Reviewer Pack — Minimal Distance between Matroid Representations and Monoton…

Entry 61283d86ec19 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 18:22:21 UTC

Minimal Distance between Matroid Representations and Monotone Circuit Size for k-CLIQUE

Entry ID: `61283d86ec19` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 18:22:21 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

> ['For any matroid M with n elements, the minimum distance between two distinct representations of M in a representation system R is Ω(2^(n/4) / log(n)) compared to the size of the smallest monotone circuit computing k-CLIQUE.']

Rationale (proposer's reasoning):

> ['Matroids encode combinatorial structures and have been used in complexity theory for their ability to represent combinatorial problems. The minimum distance between representations could provide a new perspective on monotone circuits, potentially leading to lower bounds for the size of monotone circuits computing k-CLIQUE.']

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7522573671a795ee`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for 30 random seeds of matroids with n elements (n ≤ 40), the ratio of the minimum distance between two distinct representations to the size of the smallest monotone circuit computing k-CLIQUE is at least Ω(2^(n/4) / log(n)). The conjecture is falsified if this ratio falls below Ω(2^(n/4) / log(n)) for any seed.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Matroid Theory" AND "Monotone Circuit Complexity" AND "k-CLIQUE" AND minimal distance` - `"representations of Matroids" AND monotone circuits AND complexity analysis` - `"minimum distance" in matroid representations AND monotone circuit size for k-CLIQUE`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_matroid(n):
        matroid = set()
        for i in range(n):
            matroid.add(frozenset(random.sample(range(1, n+1), i+1)))
        return matroid

    def min_distance(matroid):
        distances = {}
        for m1 in matroid:
            for m2 in matroid:
                if m1 != m2:
                    distance = sum(1 for x in m1 if x not in m2)
                    distances[(m1, m2)] = distance
                    distances[(m2, m1)] = distance
        return min(distances.values())

    def monotone_circuit_size(n):
        # This is a placeholder function. Implement the actual circuit size calculation.
        return 2**n

    n = random.randint(5, 40)
    matroid = generate_matroid(n)
    min_dist = min_distance(matroid)
    circuit_size = monotone_circuit_size(n)

    if circuit_size == 0:
```

```
        return {
            "metric_name": "Ratio of min distance to circuit size",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Circuit size is zero"
        }

    ratio = min_dist / circuit_size
    conjecture_holds = ratio >= (2**(n/4) / math.log(n))
    counterexample = f"Ratio {ratio} < Ω(2^{n/4}/log({n})) for n={n}" if not conjecture_holds else

    return {
        "metric_name": "Ratio of min distance to circuit size",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
jecture_holds': False, 'counterexample': 'Ratio 0.0 < Ω(2^3.75/log(15)) for n=15'}
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
```

```
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste
TRIAL: {'metric_name': 'Ratio of min distance to circuit size', 'metric_value': 0.0, 'instances_teste

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8fa39ddb.py", line 86, in <module>
    first_failing_seed = n
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The ratio of the minimum distance between two distinct representations to the size of the smallest monotone circuit computing k-CLIQUE is below Ω(2^(n | next: Investigate the specific cases where the conjecture fails to determine if there are underlying patterns or if this is a general issue. Consider adjusting the conjecture's parameters or exploring alternative approaches.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15008
2	preregistration	ollama_remote	glm4:latest	0	0	9861
3	novelty	ollama_remote	glm4:latest	0	0	9204
4	novelty	ollama_remote	glm4:latest	0	0	8545
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13118
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8716
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8566
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9390
9	judge	ollama_remote	glm4:latest	0	0	36133

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118542 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/61283d86ec19.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/61283d86ec19.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/61283d86ec19.tar.gz` (if generated)